Task 16: Automate Deployment & Load Balancing with SSM (Flask + MongoDB Atlas)

Architecture

- **Application Layer (2 × EC2)**: Flask API servers (running via Gunicorn or systemd).
- **Load Balancer**: Application Load Balancer (ALB) distributing traffic across the two Flask EC2 instances in different Availability Zones.
- **Database Layer**: MongoDB Atlas cluster (cloud-hosted, managed backups).

Automation Flow

- **Deployment & Restart**: Managed by SSM Run Command across both Flask EC2s.
- **Database**: MongoDB Atlas is used → no EC2-based DB, no backup jobs needed.

Deployment Automation (SSM Run Command)

Create an **SSM Document** named **DeployFlaskApp**.

The document should:

1. Pull the latest Flask app code from GitHub 👉 FlaskWithMongoDB.
2. Install or update Python dependencies:

```
pip install -r requirements.txt
```

3. Restart Flask app service:

```
systemctl restart flaskapp
```

or

```
gunicorn --reload -b 0.0.0.0:5000 app:app
```

4. Log deployment output to:

```
/var/log/deploy.log
```

Run this SSM Run Command on **both EC2 instances** behind the load balancer whenever a new deployment is needed.

Load Balancer Setup

- Create an **Application Load Balancer (ALB)** in front of the Flask app instances.

- **Target Group**: Add both Flask EC2 instances (spread across different AZs).
- **Health Check**: HTTP `/` endpoint on port **5000** (or the Flask app port).
- **Security**: Allow inbound HTTP/HTTPS traffic only via ALB; block direct public access to EC2s.

Result: Requests are evenly distributed across the Flask servers.

Expected Deliverables

- **ALB + 2 Flask EC2 instances across AZs** → working load-balanced app.
- **SSM Document** → `DeployFlaskApp` for app deployment across both EC2s.
- **MongoDB Atlas cluster** linked to Flask app (connection string in `.env` or config).
- **Logs**: `/var/log/deploy.log` showing successful deployments.
- **Screenshot**: Flask app accessible via the ALB DNS name.

Time to complete: 3 hours